# QuantuML: Machine Learning Algorithms and K-means on Quantum Cloud offering

Constantinos Bitsakos[1][0009−0003−3669−0453], Konstantinos Nikas[1][0000−0003−4424−9951], and Nectarios Koziris[1][0000−0002−4890−8427]

CSLab, School of Electrical and Computer Engineering, National Technical University of Athens, Patision 47 Athens Greece
{kbitsak, knikas, nkoziris}@cslab.ece.ntua.gr

**Abstract.** Quantum Computing emerged from the pioneering ideas of Paul Benioff and Richard Feynman in the early 1980s. However, due to the technological limitations of the time, constructing a stable Quantum Computer remained infeasible, leaving Quantum Computing to theoretical and algorithmic realms. Algorithms such as Shor's and Grover's have since demonstrated the superiority of Quantum Models over Classical Models.

In recent years, technological advancements have equipped us with the means to develop stable and capable Quantum Computing Systems, exemplified by entities like Google (Quantum Supremacy), Microsoft, IBM Q, Rigetti, Honeywell, and IonQ. Many of these systems are now accessible to individual users, much like other IAAS offerings.

This work introduces the logic of Quantum Computing and delves into fundamental Quantum Algorithms to elucidate the programming patterns employed in utilizing Quantum Hardware. Subsequently, we examine several Quantum Machine Learning Algorithms. Specifically, we develop a hybrid Quantum-Classical algorithm for k-means clustering and theoretically demonstrate its supremacy over classical algorithms in terms of execution time. We validate its efficiency in a Quantum Simulation Environment.

Finally, we test the algorithm on real Quantum Hardware provided by IBMQ, drawing conclusions about the stability and efficiency of contemporary Quantum Computing Systems.

**Keywords:** Quantum Algorithms · Machine Learning · K-means · IBMQ · Qiskit.

## 1 Introduction

Quantum Computing was born from the theoretical ground work done by Paul Benioff and Richard Feynman in the 1980s [1] At the beginning of evolution of Quantum Computing, it was challenged by technological barriers and thus was purely based on the theoretical approach and algorithms. Algorithms like Shor's [2] and Grover's [3] were the pioneers that proved the effectiveness of Quantum

Models over the Classical Models but their application was rather limited at first.

Recent developments have made it quite feasible to develop Quantum Computing Systems which are stable [4]. Big players like Google, Microsoft, IBM Q, Rigetti, Honeywell, and IonQ have played a major role in this advancement. Most of these systems are available today to the general consumers via IAAS, which has made quantum computing available to the masses [5].

Some of the biggest companies that provide cloud services offer Quantum Computing services; IBM, Google, Microsoft, Amazon Web Services (AWS) and Rigetti [6]. IBM's Quantum Experience gives clients a way to work with quantum processors and code in Qiskit, an open-source quantum software development kit [7]. Quantum AI by Google allows researchers to run their experiments on the quantum processors that Google provides [4]. Microsoft Azure Quantum is the cloud service that connects quantum computing with the tools from IonQ and Honeywell. AWS Braket is a fully managed quantum computing service where the researchers can leverage the quantum devices from D-Wave, IonQ, and Rigetti. These platforms are very critical in the evolution and promotion of quantum computing.

Quantum Computing can be the new breakthrough in solving problems by providing solutions that may take classical computers decades to come up with. Shor's algorithm has exponential improvement for integer factorization where as Grover's algorithm has quadratic improvement for unstructured search. These innovations imply a fundamental change in approach, which has a great impact on such areas as cryptography, optimization, and drug design.

This paper presents the foundational principles of Quantum Computing and explores key Machine Learning Quantum Algorithms [8]. We discuss programming paradigms for effective use of Quantum Hardware and introduce a hybrid Quantum-Classical k-means [9] clustering algorithm, validated in a Quantum Simulation Environment.

**Our experiments were conducted on IBM's Quantum platform** using Qiskit in both simulation and actual execution modes. These experiments underscore the current limitations of quantum processors, primarily due to stability issues, which are detailed in this paper.

Empirical tests on IBMQ's real Quantum Hardware were conducted to assess the stability and efficiency of current Quantum Computing Systems. Our findings provide valuable insights into the state of Quantum Computing technology and its practical applications, informing future research and development.

The paper is structured as follows: Section 1 provides an introduction to the topic. Section 2 reviews related work and significant contributions. Section 3 discusses Quantum Computing and the design of our hybrid Quantum K-means algorithm. Section 4 presents the experiments and results from our simulation and hardware tests. Section 5 concludes the paper and suggests directions for future research.

This paper aims to offer a comprehensive overview of Quantum Computing's capabilities and potential, highlighting its significant advantages and fostering a deeper understanding of its impact on the future of technology.

## 2 Related Work and the History of Quantum Computing

The development of Quantum Mechanics was significantly influenced by scientists such as Niels Bohr, Louis de Broglie, Albert Einstein, Erwin Schrödinger, Werner Heisenberg, and John von Neumann, who established its theoretical and mathematical foundations [1].

Richard Feynman, Yuri Manin, and Paul Benioff presented the idea of Quantum Computer in the early 1980s based on the Classical Model of Alan Turing [10]. Shor's Algorithm proposed in 1994 was the first to show that Quantum Computers are more efficient in factoring large prime numbers [2]. In 1996, Lov Grover came up with a Quantum Search Algorithm which provided a polynomial gain over the classical search algorithms [3].

The first experimental quantum computers appeared at the late 1990s. The Technical University of Munich unveiled a 5-qubit Quantum Computer by the year 2000, and this was followed by the Los Alamos National Laboratory's 7-qubit Quantum Computer [6]. Shor's Algorithm was demonstrated by IBM in 2001 and by 2005 information was exchanged between two Quantum Memories [6].

Some of the most important milestones are the first 14-qubit register in 2011, a 300-qubit simulator in 2012 and the first silicon logic gate for 2 qubits in 2015 [11]. IBM introduced the Quantum Experience in 2016 which is an entry point to Quantum Computing Systems for the public [5]. IBM created a 53-qubit Quantum Computer in 2019 and Google claimed Quantum Supremacy in the same year [4].

Despite these advancements, publicly accessible Quantum Computers with consistent performance surpassing classical models are still in development [12]. Quantum-enhanced machine learning leverages qubits and quantum operations to enhance computational speed and data storage, combining classical and quantum processing for more efficient execution [8].

Quantum machine learning involves applying classical methods to data from quantum experiments, understanding phase transitions in quantum systems, and devising new quantum experiments. Research also investigates the similarities between physical systems and learning systems, especially neural networks, where quantum physics techniques apply to classical deep learning and vice versa [13]. Quantum learning theory explores abstract concepts of learning theory within quantum information, contributing to a deeper understanding of learning systems in quantum environments [8].

# 3 Quantum Computing and the Design of our Hybrid Quantum K-means Algorithm

## 3.1 Introduction to Qubits

In quantum computing, the basic unit of information is the qubit 1. Unlike classical bits, which can be either 0 or 1, a qubit can exist simultaneously in multiple states through superposition, allowing quantum computers to perform complex computations more efficiently [14].

A qubit is represented as a combination of two states, 0 and 1. Using Dirac notation, we have

$$|\Psi\rangle = \alpha\,|0\rangle + \beta\,|1\rangle$$

where $\alpha$ and $\beta$ are the amplitudes of each state.

$$|0\rangle = \begin{bmatrix}1\ 0\end{bmatrix} \quad |1\rangle = \begin{bmatrix}0\\1\end{bmatrix}$$

$$|+\rangle = \tfrac{1}{\sqrt{2}}\,|0\rangle + \tfrac{1}{\sqrt{2}}\,|1\rangle \quad |-\rangle = \tfrac{1}{\sqrt{2}}\,|0\rangle - \tfrac{1}{\sqrt{2}}\,|1\rangle$$

$$|i\rangle = \tfrac{1}{\sqrt{2}}\,|0\rangle + \tfrac{i}{\sqrt{2}}\,|1\rangle \quad |-i\rangle = \tfrac{1}{\sqrt{2}}\,|0\rangle - \tfrac{i}{\sqrt{2}}\,|1\rangle$$

Fundamental Qubit states

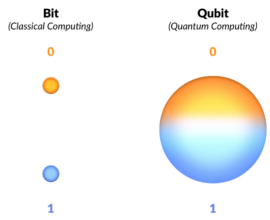



Fig. 1

## 3.2 Quantum Entanglement

Quantum Entanglement refers to the phenomenon where two particles, regardless of their distance, interact so that any change in one instantly affects the other. This defies conventional limits, like the speed of light. The particles remain "connected" regardless of distance, meaning information is transferred instantaneously [15].

In Quantum Computers, entangled qubits exhibit this phenomenon. Any change in one qubit affects the other. This can be illustrated as:

$$|\Psi\rangle = \frac{1}{2}|aa\rangle + \frac{1}{2}|ba\rangle + \frac{1}{2}|ab\rangle + \frac{1}{2}|bb\rangle$$

$$|\Phi\rangle = \frac{1}{\sqrt{2}}|aa\rangle + \frac{1}{\sqrt{2}}|bb\rangle$$

| Possible outcome of state $\|\psi\rangle$ | | |
|---|---|---|
| Particle 1: $\|a\rangle$ | $\|\psi\rangle \Rightarrow \|aa\rangle : \frac{1}{4}$ | $\|\psi\rangle \Rightarrow \|ab\rangle : \frac{1}{4}$ |
| Particle 2: $\|b\rangle$ | $\|\psi\rangle \Rightarrow \|ba\rangle : \frac{1}{4}$ | $\|\psi\rangle \Rightarrow \|bb\rangle : \frac{1}{4}$ |

| Possible outcome of state $\|\phi\rangle$ | | |
|---|---|---|
| Particle 1: $\|a\rangle$ | $\|\phi\rangle \Rightarrow \|aa\rangle : \frac{1}{2}$ | $\|\phi\rangle \Rightarrow \|ab\rangle : 0$ |
| Particle 2: $\|b\rangle$ | $\|\phi\rangle \Rightarrow \|ba\rangle : 0$ | $\|\phi\rangle \Rightarrow \|bb\rangle : \frac{1}{2}$ |

For superposition state $\Psi$, there is a 50% chance of finding the first qubit in state $a$, and a 25% chance of the second qubit being in state $a$ or $b$. For superposition state $\Phi$, if the first qubit is in state $a$, the second qubit will definitely be in state $a$ and never in state $b$. This paradox is utilized in Quantum Computing [10].

### 3.3 Quantum Gates and Circuits

In Quantum Computers, we program at the level of logic gates [16]. Quantum Logic Gates must:

- Preserve the sum of the square norm of the initial superposition.
- Be reversible, allowing the system to revert to its initial state if the same gate is applied again [17].

Each Quantum Gate is represented by an $n \times n$ Unitary Matrix:

$$U^*U = UU^* = I$$

To apply a Quantum Gate, we find the tensor product of the gate with the system [18].

**Important Quantum Gates** The Identity Gate outputs the same unaltered system.

$$I = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

The NOT gate inverts all its qubits:

$$\text{NOT} = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$$

$$\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \end{bmatrix} \qquad \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

The Hadamard Gate produces a system with each state equally probable:

$$H|0\rangle = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \frac{|0\rangle + |1\rangle}{\sqrt{2}}$$

$$H|1\rangle = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ -1 \end{bmatrix} = \frac{|0\rangle - |1\rangle}{\sqrt{2}}$$

The Swap Gate swaps qubit positions.

$$SWAP = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$SWAP|01\rangle = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \end{bmatrix} = |10\rangle$$

Using Quantum gates like Toffoli, Pauli, Phase Shifting Gates, and Rotation Operator Gates [17], we construct Quantum Circuits [18] that manipulate inputs from the superposition of many qubits to perform desired operations.

### 3.4 Quantum Algorithms

Quantum Algorithms are probabilistic. The output after one execution is not guaranteed to be correct. **The execution must be repeated many times to reach a reliable conclusion,** adding to the time complexity. Problems solvable with Quantum Programming are **BQP (bounded error, quantum, polynomial time)** [19]. These problems can be solved in polynomial time with a maximum error probability of 1/3.

The class **BQP (bounded error, quantum, polynomial time)** is the quantum equivalent of **BPP (bounded error, probabilistic, polynomial time)** [20] in Classical Computing. It is proven that **BPP (bounded error, probabilistic, polynomial time)** $\subseteq$ **BQP (bounded error, quantum, polynomial time)**, and it is conjectured that **BQP (bounded error, quantum, polynomial time)** $\not\subseteq$ **BPP (bounded error, probabilistic, polynomial time)**.

We do not know the exact position of **BQP (bounded error, quantum, polynomial time)** in **PSPACE** [21], but we know that **P** $\subseteq$ **BQP (bounded error, quantum, polynomial time)** $\subseteq$ **PSPACE**. **BQP (bounded error, quantum, polynomial time)** is conjectured to be within **PSPACE**.

In quantum algorithms, we need a **Quantum Oracle** [22], a quantum function that must be reversible, combining with the desired function.

### 3.5 A Quantum Kmeans approach

The k-means algorithm [9] clusters a dataset into $k$ subgroups based on the mean of their distance from $k$ centroids.

The k-means algorithm is simple in design (Algorithm 1):

**Algorithm 1** k-means Algorithm

1: Determine number of centroids (K)
2: Randomly select K distinct centroids
3: Measure the distance (Euclidean distance) between each point and the centroid
4: Assign each point to the nearest cluster
5: Calculate the mean of each cluster as the new centroid
6: Repeat steps 3–5 with the new center of the cluster
7: Calculate the variance of each cluster
8: Repeat steps 2–7 until the lowest sum of variance is achieved

**Swap Test** The first step in presenting Quantum k-means is the Swap Test routine [23], a well-known routine in Quantum Programming that provides an estimate of the "similarity" between two states, i.e., the overlap between two states $\langle a|b\rangle$. The algorithm is as follows (Algorithm 2):

**Algorithm 2** Swap Test Algorithm

1: Initialize two states $|a\rangle, |b\rangle$ of $n$ qubits each and a control qubit $|0\rangle$: $|\psi_0\rangle = |0, a, b\rangle$
2: Apply Hadamard gate on control qubit: $|\psi_1\rangle = H \otimes I^{\otimes n} I^{\otimes n} |\psi_0\rangle = \frac{1}{\sqrt{2}}(|0, a, b\rangle + |1, a, b\rangle)$
3: Apply controlled SWAP gate on $|a\rangle, |b\rangle$ that swaps $a, b$ if the control qubit is 1: $|\psi_2\rangle = \frac{1}{\sqrt{2}}(|0, a, b\rangle + |1, b, a\rangle)$
4: Apply second Hadamard gate on control qubit: $|\psi_3\rangle = \frac{1}{2}|0\rangle(|a, b\rangle + |b, a\rangle) + \frac{1}{2}|1\rangle(|a, b\rangle - |b, a\rangle)$
5: Perform Measurement:

$$\begin{aligned} P(|0\rangle) &= \left|\frac{1}{2}\langle 0|0\rangle(|a, b\rangle + |b, a\rangle) + \frac{1}{2}\langle 0|1\rangle(|a, b\rangle - |b, a\rangle)\right|^2 \\ &= \frac{1}{4}|(|a, b\rangle + |b, a\rangle)|^2 \\ &= \langle b|b\rangle\langle a|a\rangle + \langle b|a\rangle\langle a|b\rangle + \langle a|b\rangle\langle b|a\rangle + \langle a|a\rangle\langle b|b\rangle \\ &= \frac{1}{2} + \frac{1}{2}|\langle a|b\rangle|^2 \end{aligned}$$

What do we observe from Algorithm 2? In the end, we get a real number resulting from the inner product of $|a\rangle, |b\rangle$. If our measurement returns 0.5, the two states have a zero inner product, meaning they are orthogonal. If it returns 1, the two states are identical. The range between 0.5-1 provides a criterion for the similarity of the two states.

We use this property to find the equivalent of the Euclidean Distance [24] in the Quantum field. If we can find the similarity between two states, we need to

model two points in the Cartesian system so that their similarity in Quantum representation reflects their proximity in the Cartesian system.

**Quantum Representation of a Classical State** To represent Cartesian points in the Quantum Model, we propose the following transformation that maps these points onto the Bloch Sphere 1, maintaining distance properties:

$$\theta_1 = (x_0 + 1)\frac{\pi}{2}, \phi_1 = (x_1 + 1)\frac{\pi}{2}, \theta_2 = (\psi_0 + 1)\frac{\pi}{2}, \phi_2 = (\psi_1 + 1)\frac{\pi}{2}$$

$$\Rightarrow |x\rangle = \left[\theta_1\ \phi_1\right], |\psi\rangle = \left[\theta_2\ \phi_2\right]$$

To bring the system to the desired Quantum state, we use Quantum Rotation Gates as shown in the implementation in the next chapter. For multi-dimensional datasets, the work in [25] achieves Quantum representation for multi-dimensional input.

Finally, the routine for time complexity calculates the similarity or distance between two points. In our implementation, it is $O(\log N)$ (because we need $\log N$ Qubits for $n$ input features), while in the Classical equivalent, the complexity is $O(n)$.

## 4 Evaluation

At this point, we demonstrate the implementation of the Quantum k-means algorithm which we run on IBM's Qiskit on the cloud [5]. Figure 2 shows the created circuit. Initially, the angles for transforming classical points onto the Bloch Sphere are computed. Using the phase shifting gate U, we bring the qubits to the appropriate position [26]. We apply the swap test and measure the qubits to find the distance or similarity of their states [23].

We execute the code as many times as specified by the parameter shots. Quantum measurement is probabilistic, so we run a routine multiple times and take the average of the results [10]. We observe the similarity results of two states for different numbers of shots in Figures 3, 3a, 3b, 3c.

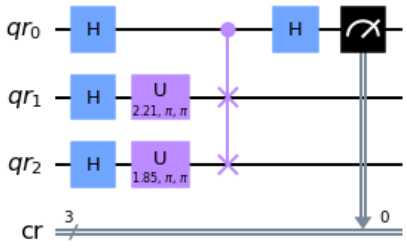
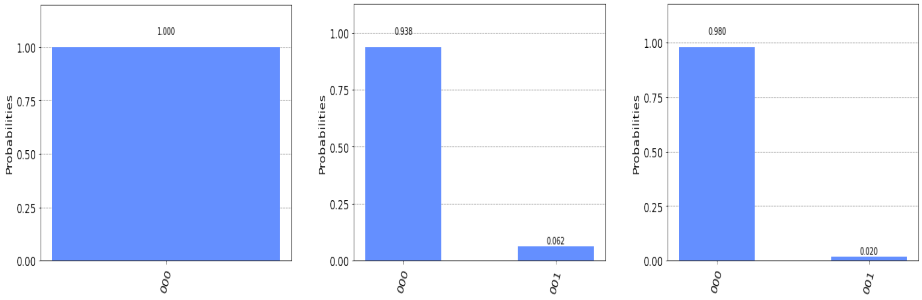


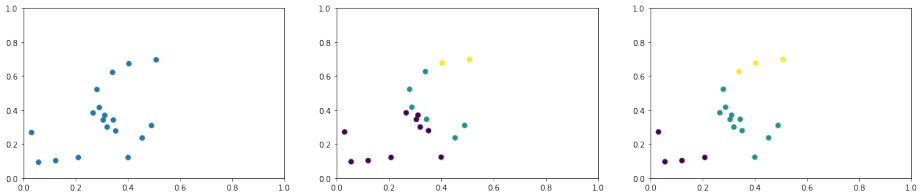

Fig. 2: Quantum distance of 2 qubits circuit




(a) (b) (c)

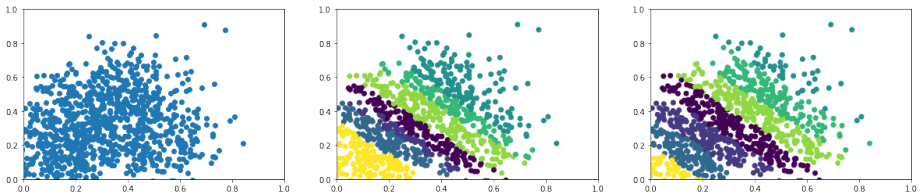
Fig. 3: Similarity of 2 states depending on the number of shots (1, 20, 100)

Next, we examine the correctness of our algorithm for clustering different numbers of input points for 7 centroids. Results over time are shown in Figures 4, 4a, 4b, 4c, and for 1000 points in Figures 5, 5a, 5b, 5c. We observe that the algorithm and Quantum representation of the Classical states work correctly and produce the desired results.



(a) (b) (c)

Fig. 4: Clustering of 20 points over time



(a) (b) (c)

Fig. 5: Clustering of 1000 points over time

**Performance evaluation of Our Algorithm's Performance** To measure the execution time with different parameters, we use the Qiskit time taken() function. We use the backend IBM boqota and an Nvidia Geforce 1650 Ti GPU for the classical k-means. Features of interest are the number of shots and input features. We assign random features to points and compute the similarity measure to see how execution time on Quantum hardware is affected by the input dimensions [5].

Finally, we create a dataset 6a with Qtime and Ctime, the execution times on Quantum and Classical models, and features and shots, the input features of k-means and the number of executions performed on Quantum hardware for reliable results.

From previous experiments, we know that execution time on Quantum hardware will be much greater than Classical time. We want to analyze if there is any correlation between input size and execution time. First, we construct a correlation matrix [27] using the Pearson method, which we present in Figure 6b. We observe a moderate to weak correlation between input feature size and number of executions on Quantum hardware with execution time. Then we perform an analysis using the Spearman method, which we present in Figure 6c, which shows correlations between two variables that are not just linear. Here, we see a strong correlation between execution time and the number of executions on Quantum hardware.

| | Qtime | Ctime | features | shots |
|---|---|---|---|---|
| 0 | 6.627725 | 0.000197 | 2.0 | 4.0 |
| 1 | 7.054593 | 0.000147 | 2.0 | 8.0 |
| 2 | 6.698065 | 0.000147 | 2.0 | 16.0 |
| 3 | 6.829511 | 0.000149 | 2.0 | 256.0 |
| 4 | 6.858196 | 0.000152 | 2.0 | 512.0 |

(a) Final Dataset

| | Qtime | Ctime | features | shots |
|---|---|---|---|---|
| Qtime | 1.000000 | 0.046473 | 0.149386 | 0.324195 |
| Ctime | 0.046473 | 1.000000 | 0.648112 | 0.204988 |
| features | 0.149386 | 0.648112 | 1.000000 | 0.377049 |
| shots | 0.324195 | 0.204988 | 0.377049 | 1.000000 |

(b) Pearson correlation

| | X | Y | method | alternative | n | r | CI95% | p-unc | power |
|---|---|---|---|---|---|---|---|---|---|
| 0 | Qtime | Ctime | spearman | two-sided | 102 | 0.108 | [-0.09, 0.3] | 0.279 | 0.192 |
| 1 | Qtime | features | spearman | two-sided | 102 | 0.262 | [0.07, 0.43] | 0.008 | 0.763 |
| 2 | Qtime | shots | spearman | two-sided | 102 | 0.435 | [0.26, 0.58] | 0.000 | 0.997 |
| 3 | Ctime | features | spearman | two-sided | 102 | 0.357 | [0.17, 0.52] | 0.000 | 0.961 |
| 4 | Ctime | shots | spearman | two-sided | 102 | 0.273 | [0.08, 0.44] | 0.006 | 0.799 |
| 5 | features | shots | spearman | two-sided | 102 | 0.204 | [0.01, 0.38] | 0.040 | 0.542 |

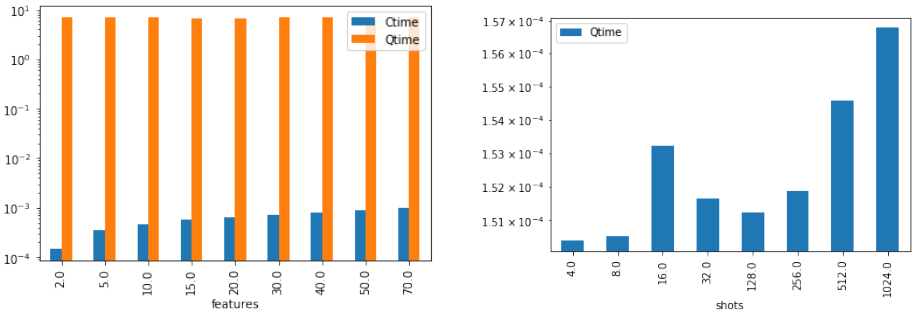
(c) Spearman correlation

Fig. 6: Analysis results

We present a chart showing the correlation between input feature size and execution time in Quantum and Classical models in Figure 7a. Since there is a large difference in magnitude, we normalize the values for Ctime. We observe that increasing input size in the Classical model results in a significant increase in

execution time, but not in the Quantum model. This indicates that the Quantum algorithm is not as affected by input size since its complexity is $O(\log(n))$.

Finally, Figure 7b illustrates the relationship between execution time on Quantum hardware and the number of executions needed for reliable results 7b. More executions lead to longer execution times.




(a) Features - Ctime and Qtime

(b) Shots and Qtime

Fig. 7: Comparison of Features and Shots with Execution Times

## 5 Conclusions and Future Work

We can safely say that the quantum hardware available for widespread use is not yet reliable in terms of execution time. However, the superiority of Quantum Algorithms in terms of time complexity is evident even on this hardware.

The primary reason for longer execution times on quantum hardware is decoherence [12]. Despite advancements providing cleaner qubits, better logic gates, and smarter algorithms, decoherence remains a major obstacle to having cheap, accessible Quantum Computers. Quantum logic gates introduce error due to decoherence, where the Quantum model interacts with the environment, causing qubits to change state uncontrollably, leading to information loss.

Due to this phenomenon, we see longer execution times in public Quantum Computers, as mitigating it with cheap methods adds to execution time. Decoherence is due to factors like magnetic or electric fields, environmental radiation, or qubit interaction problems.

With more stable Quantum Computers, we will see tremendous acceleration in many algorithms. Some scientists argue that measuring execution time with time taken is inaccurate and suggest a system-agnostic method for better speedup measurement for Quantum and Classical hardware.

The metric based on CLOPS (Circuit Layer Operations Per Second) [28] has been proposed. It measures how much work a Quantum system performs in a specific unit of time, directly related to how quickly a Quantum system executes a specific Quantum circuit. It considers factors like:

- Repeatability of a process on the Quantum processor
- Speed of logic gates
- Compilation time at runtime
- Time to generate Classical control commands
- Time to transfer information and data between Quantum units

**Future work** In continuation of this work, it is planned to carry out a more detailed analysis of the hybrid k-means proposed, so that more layers of the algorithm work at the Quantum level and it can take inputs of higher dimensions, which is possible through the Quantum representation of larger classical states [29]. Also, being able to run in larger state spaces, for example, using more than 5 qubits, such as in the IBM Quantum Researchers Program, especially for researchers and institutions, where we can also get microwave pulse control for qubits is important. The integration of the two algorithms used in a problem like Time Series Clustering, the analysis of CLOPS and the measurement of the execution time using them as a unit of measure, and the analysis of Quantum Algorithms for Neural Networks, especially Bayesian Deep Learning [13], are significant advances.

Moreover, it is necessary to integrate Quantum Neural Networks for Deep Reinforcement Learning with Double Deep Q Networks as it was done in our previous works in [30] [13]. These efforts will help improve the applicability and efficiency of Quantum Computing in different sectors, and thus contribute to the development of new technologies and discoveries.

## References

1. George Temple. *The General Principles of Quantum Theory*. Routledge, 2014.
2. Peter W. Shor. Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. *SIAM Journal on Computing*, 26(5):1484–1509, 1997.
3. Lov K. Grover. A fast quantum mechanical algorithm for database search, 1996.
4. Frank Arute et al. Quantum supremacy using a programmable superconducting processor. *Nature*, 574(7779):505–510, 2019.
5. Ibm quantum experience.
6. Quantum providers.
7. Qiskit aer.
8. Dawid Kopczyk. Quantum machine learning for data scientists, 2018.
9. Leonard Schulman et al. A theoretical analysis of lloyd's algorithm for k-means clustering. 2022.
10. Anne-Catherine de la Hamette et al. Falling through masses in superposition: quantum reference frames for indefinite metrics, 2021.
11. A. M. J. Zwerver et al. Qubits made by advanced semiconductor manufacturing. *Nature Electronics*, 5(3):184–190, 2022.
12. Encoding of classical data to quantum. *Towards Data Science*.
13. Patrick Rebentrost Peter Wittek Zhikuan Zhao, Alejandro Pozas-Kerstjens. Bayesian deep learning on a quantum computer. *Quantum Machine Intelligence*, 1(1-2):41–51, 2019.

14. Isaac L. Chuang Michael A. Nielsen. *Quantum Computation and Quantum Information: 10th Anniversary Edition*. Cambridge University Press, 2011.
15. Ryszard Horodecki et al. Quantum entanglement. *Reviews of Modern Physics*, 81(2):865–942, 2009.
16. Lin Weng. Local and global quantum gates, 2021.
17. Most famous quantum gates.
18. Noam Nisan Dorit Aharonov, Alexei Kitaev. Quantum circuits with mixed states. In *Proceedings of the Thirtieth Annual ACM Symposium on Theory of Computing*, pages 20–30, 1998.
19. Adam Bouland. *Space around bounded-error quantum polynomial time*. PhD thesis, 2017.
20. Christos H. Papadimitriou. *Computational Complexity*, pages 260–265. John Wiley and Sons Ltd., 2003.
21. Adi Shamir. Ip = pspace. *J. ACM*, 39(4):869–877, 1992.
22. Liyi Li et al. Verified compilation of quantum oracles, 2021.
23. Min-Sung Kang et al. Implementation of swap test for two unknown states in photons via cross-kerr nonlinearities under decoherence effect. *Scientific reports*, 9(1):6167–6167, 2019.
24. Kenneth J. Berry Paul W. Mielke. *Euclidean Distance Based Permutation Methods in Atmospheric Science*, pages 7–27. Springer US, 2000.
25. Encoding of classical data to quantum. *Towards Data Science*.
26. Chu-Ryang Wie. Bloch sphere model for two-qubit pure states, 2014.
27. W. Alan Nicewander Joseph Lee Rodgers. Thirteen ways to look at the correlation coefficient. *The American Statistician*, 42(1):59–66, 1988.
28. Andrew Wack et al. Quality, speed, and scale: three key attributes to measure the performance of near-term quantum computers, 2021.
29. Steven Herbert. Increasing the classical data throughput in quantum networks by combining quantum linear network coding with superdense coding. *Phys. Rev. A*, 101(6):062332, 2020.
30. Nectarios Koziris Constantinos Bitsakos, Ioannis Konstantinou. Derp: A deep reinforcement learning cloud system for elastic resource provisioning. In *2018 IEEE International Conference on Cloud Computing Technology and Science (CloudCom)*, pages 21–29, 2018.